



Carátula para entrega de prácticas

Laboratorio de computación salas A y B

Profesor: M.C Rene Adrian Davila Perez

Asignatura: Programación orientada a objetos

Grupo: 7

*No de
Práctica(s):* 5

Integrante(s): 323036285
323279866
323209432
323161891
323063609

*No. De Lista o
brigada* 3

Semestre: 2027-1

*Fecha de
entrega:* 2/10/2026

Obervaciones:

CALIFICACIÓN: _____

Índice

1. Introducción	2
2. Marco Teórico	3
2.1. Atributos [3]	3
2.2. Modificadores [2]	3
2.2.1. Tipos de modificadores [2]	3
2.3. Constructores [1]	4
2.3.1. Tipos de constructores [1]	4
3. Desarrollo	5
4. Resultados	10
5. Conclusión	15

1. Introducción

Actualmente los alumnos de diversas disciplinas recurren a los modelos de lenguaje de gran escala para resolver dudas técnicas sobre programación. Sin embargo, esto trae consigo un inconveniente importante: muchos adoptan estas respuestas de forma acrítica, ignorando si son verdaderas. Como estas herramientas suelen cruzar o confundir sintaxis y lógicas entre distintos lenguajes, la calidad y la fiabilidad del proceso de aprendizaje terminan viéndose comprometidas.

Al mismo tiempo, al iniciar en un nuevo lenguaje de programación al estudiantado se le presentan problemas que tienen múltiples soluciones, lo cuál se vuelve un problema a la hora de programar, ya que si no se manejan correctamente las estructuras más simples del lenguaje, un programa muy simple puede terminar siendo mucho más complejo de lo que se esperaba.

Resolver estos problemas nos ayudará a usar mejor a los LLM a la hora de consultar conceptos técnicos, consiguiendo una mejor comprensión y más veracidad en nuestro aprendizaje, también nos ayudará a comprender mejor el lenguaje de programación Java, y a practicar para encontrar las soluciones más simples y óptimas.

Objetivos

- Preguntarle a un LLM si es conveniente usar métodos modificadores en los constructores, para verificar su información y discernir si está en lo correcto y nos da la mejor solución.
- Con base en la respuesta del modelo de lenguaje, elaborar dos programas usando modificadores, para aprender a emplearlos y hacer correcto uso de ellos.

2. Marco Teórico

En esta práctica se tomaron en cuenta los conceptos de atributos, modificadores y constructores en Java.

2.1. Atributos [3]

Son aquellas variables declaradas dentro de una clase.

2.2. Modificadores [2]

Son palabras clave que se utilizan para cambiar las propiedades de clases, métodos y variables.

Definen la accesibilidad, el comportamiento y el alcance de sus componentes.

Están divididos en modificadores de acceso y no acceso.

2.2.1. Tipos de modificadores [2]

- **De acceso:**

- **Público:** El miembro es accesible desde cualquier clase.
- **Protegido:** El miembro sólo es accesible dentro de su propio paquete y por las subclases.
- **Privado:** El miembro sólo es accesible dentro de su propia clase.
- **Por defecto:** (Sin palabra clave) El miembro sólo es accesible dentro de su propio paquete.

- **Sin acceso:**

- **Estático:** El miembro pertenece a la clase, no a sus instancias.

- **Final:** Para declarar constantes, evita la sobreescritura de métodos y la herencia.
- **Abstracto:** Declara clases y métodos abstractos.

2.3. Constructores [1]

Son métodos utilizados para inicializar objetos, se invocan al instanciar una clase, comparten su mismo nombre y no tienen tipo de retorno.

Se pueden utilizar para evitar la duplicación de código llamando a un constructor desde otro dentro de la misma clase utilizando `this()`.

2.3.1. Tipos de constructores [1]

- **Por defecto:** Se proporciona automáticamente si no se definen explícitamente en la clase, inicializa el objeto con valores por defecto.
- **Sin argumentos:** Definido por el programador, no recibe parámetros y puede incluir código de inicialización personalizado.
- **Parametrizados:** Acepta argumentos para inicializar un objeto con valores específicos.

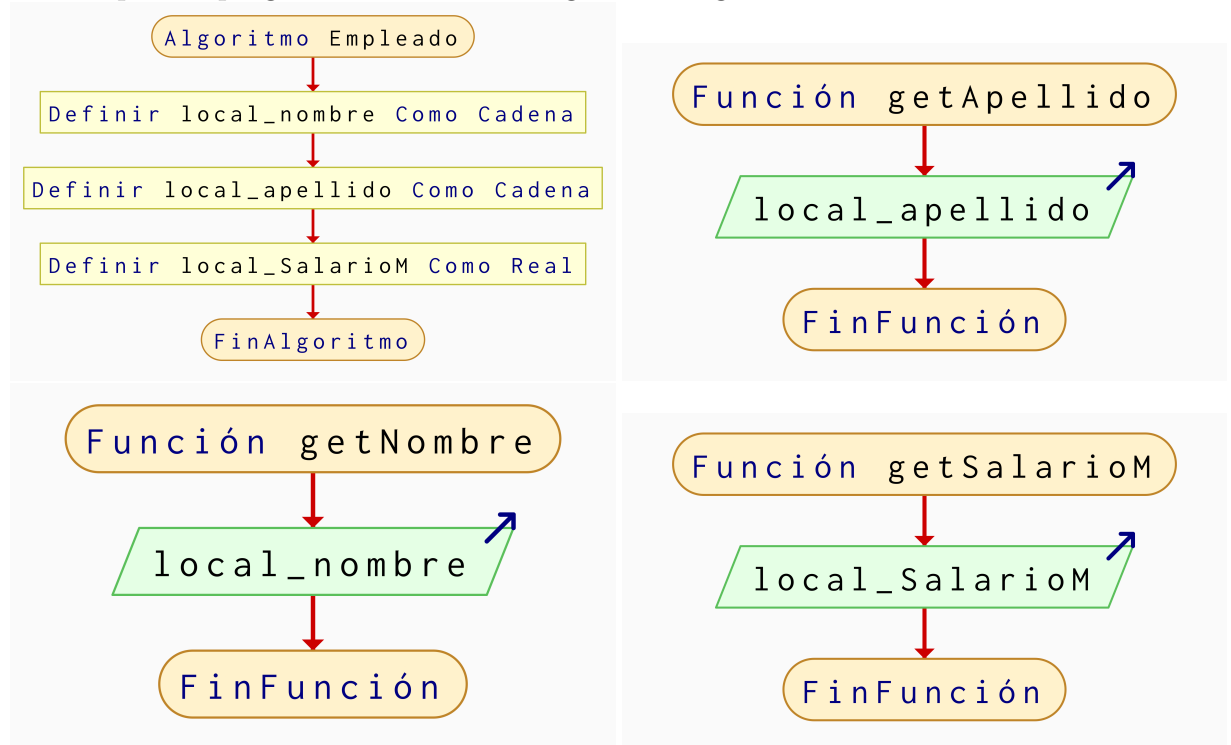
3. Desarrollo

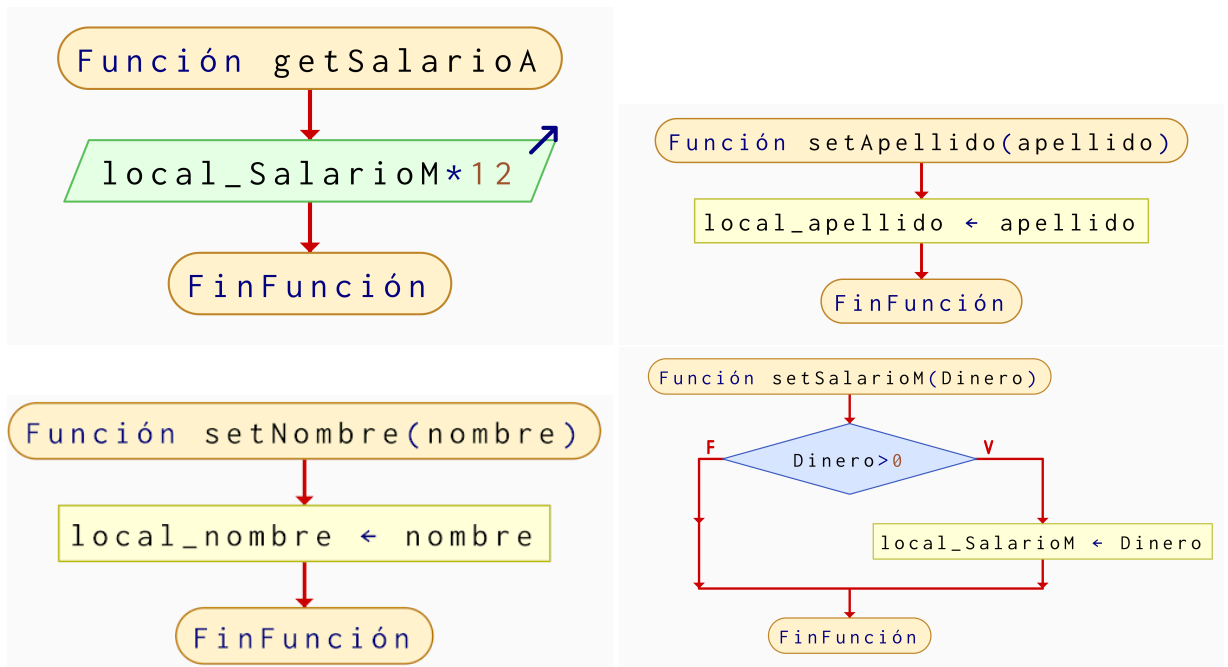
Ejercicio: Modificadores en constructores según la IA

Para el primer ejercicio, le preguntamos a nuestro LLM, acerca de si es conveniente usar modificadores en los constructores, y con su respuesta se continuó al segundo ejercicio

Ejercicio: Empleado

Para el primer programa se usaron los siguiente diagramas:





Clase Empleado

Se empezó haciendo una clase llamada **Empleado**, con atributos `nombre` y `apellido` de tipo `string`, y `salarioMensual` de tipo `double`, todos encapsulados de forma privada.

Posteriormente, se siguió con el constructor, donde se le asigna un valor a cada atributo, en cuanto al atributo `salarioMensual`, éste deberá ser mayor a 0. En la siguiente parte se procedió con los **setters**, creando uno para `nombre`, uno para `apellido` y uno para `suelo`. Finalmente se programaron los **getters** para cada atributo, los métodos retornan lo que esté guardado en ese atributo.

Clase MainEmpleado

La clase **MainEmpleado** se ejecuta en el método estático `main`. Se importó el paquete `java.util.scanner`.

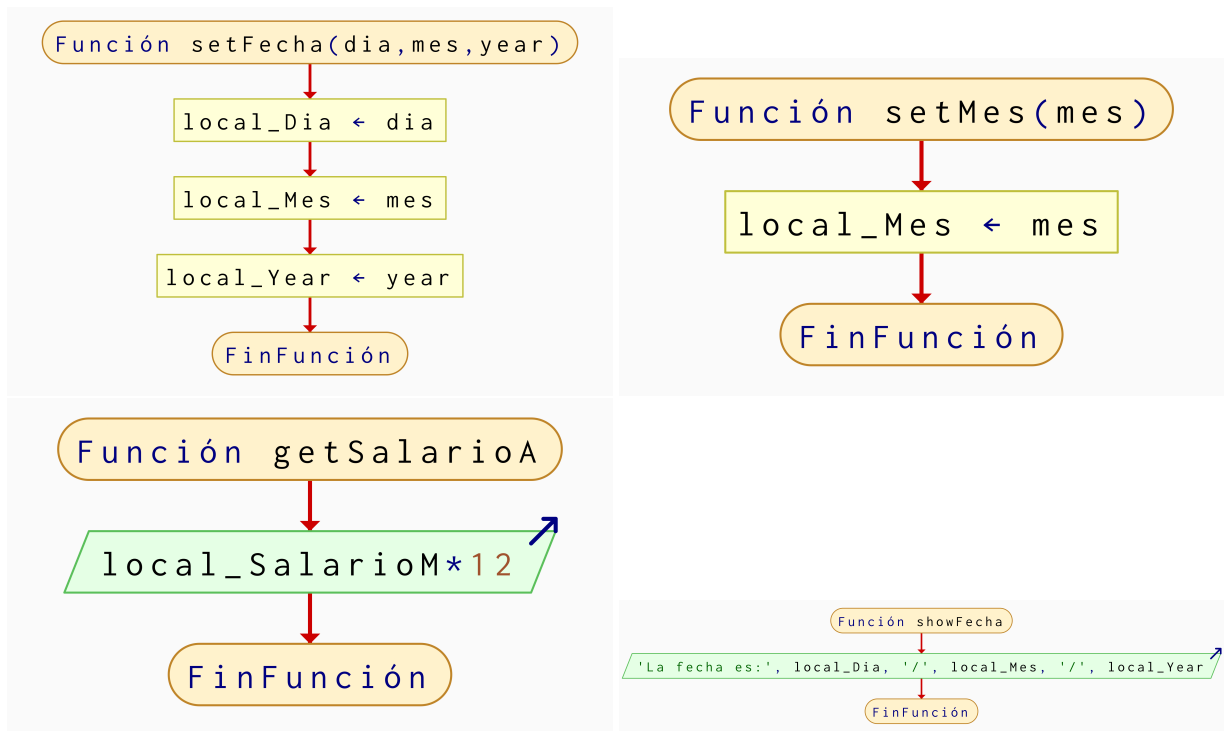
Dentro de la clase `main` se creó una variable local de tipo `scanner`, donde se guarda lo que ingresa el usuario. Después se procedió con la parte con la que el usuario interactúa, donde con un `scanner` se lee lo que irá en los atributos `nombre`, `apellido` y `salarioMensual` de la clase `Empleado`; con un constructor se crea un objeto con esos datos, este proceso se repite para dos empleados.

Posteriormente se programó la parte donde se imprimen los datos de los empleados, donde se usan `getters` para mostrar en pantalla la información. Finalmente se estableció que con ayuda de un `set` se le subiría el sueldo a cada empleado, y tras esta operación se imprimen los nuevos datos.

Ejercicio: Fecha

Para el segundo programa se usaron los siguientes algoritmos:





Clase Fecha

Se empezó haciendo una clase llamada `fecha`, con atributos `mes`, `dia` y `año` de tipo `int`, todos encapsulados de forma privada.

Posteriormente, se siguió con el constructor, donde se le asigna un valor a cada atributo. En la siguiente parte se procedió con los **setters**, creando uno para `mes`, uno para `día` y uno para `año`. Posteriormente se programaron los **getters** para cada atributo, los métodos retornan lo que esté guardado en ese atributo. Finalmente se creó un método para imprimir la fecha.

Clase MainFecha

La clase `MainFecha` se ejecuta en el método estático `main`. Se importó el paquete `java.util.scanner`.

Dentro de la clase `main` se creó una variable local de tipo `scanner`, donde se guarda lo que ingresa el usuario. Después se procedió con la parte con la que el usuario interactúa, donde con un `scanner` se lee lo que irá en los atributos `día`, `mes` y `año` de la clase `fecha`; con un constructor se crea un objeto con esos datos, para posteriormente imprimir la fecha empleando el método `mostrarFecha`.

Después, con ayuda de `setters`, se edita la fecha y finalmente se imprime la nueva fecha.

4. Resultados

Ejercicio 1

En el primer ejercicio se le presentó a nuestra IA generativa el siguiente prompt:

¿Es conveniente usar métodos modificadores en los constructores? Justificación de al menos media cuartilla

A lo que respondió:

¿Es conveniente usar métodos modificadores en los constructores?

No, no es conveniente ni una buena práctica invocar métodos modificadores (o de mutación) desde el constructor de una clase. Aunque sintácticamente el compilador lo permite, esta técnica contraviene principios fundamentales de la Programación Orientada a Objetos (POO) y genera riesgos significativos en la mantenibilidad y estabilidad del software.

A continuación se desarrollan las principales razones conceptuales y arquitectónicas que justifican evitar esta práctica:

1. Rompimiento del principio de inicialización completa e inmutabilidad

El propósito principal de un constructor es asignar un estado inicial válido, coherente y predecible al objeto que apenas se está instanciando en la memoria RAM.

Cuando un objeto es creado a través de `new`, este existe en un estado de transición durante la ejecución del constructor. Si dentro de ese constructor se invocan métodos modificadores (como `setters` o métodos de actualización

interna), se asume implícitamente que el objeto ya se encuentra completamente construido y listo para operar. Si dichos métodos hacen referencia a otros atributos del objeto que aún no han sido asignados por el flujo natural del constructor, se pueden ocasionar inconsistencias de estado o excepciones de tipo `NullPointerException`.

2. Vulnerabilidad ante la sobrescritura y el polimorfismo (Invocación de métodos sobreescribibles)

El riesgo más severo al llamar métodos de instancia desde un constructor ocurre cuando la clase no es declarada como final. Si un método modificador invocado dentro del constructor es de tipo público o protegido, este puede ser sobrescrito (`overridden`) por una subclase que herede de la actual.

En Java, cuando se crea una instancia de una subclase, el constructor de la clase madre (`superclass`) se ejecuta en primer lugar. Si este llama a un método modificador que la subclase ha modificado, se ejecutará el código de la subclase antes de que el constructor de dicha subclase haya tenido la oportunidad de inicializar sus propios atributos. Esto deriva en errores sutiles de ejecución en los que la subclase intenta operar sobre miembros que aún valen `null` o `0`.

3. Falta de claridad en la responsabilidad y violación de encapsulamiento

Un constructor debe enfocarse en la creación, mientras que los métodos modificadores están diseñados para la manipulación de comportamiento posterior a la creación. Mezclar ambos conceptos genera acoplamiento innecesario.

Si las validaciones o transformaciones simples que realiza un setter son indispensables durante la construcción, lo ideal es:

- Asignar las variables directamente dentro del cuerpo del constructor (`this.atributo = valor;`).

- Extraer la lógica de validación o transformación a métodos auxiliares privados o estáticos (`private` o `static`), de modo que no puedan ser sobrescritos externamente ni dependan del estado mutable del objeto actual.

4. Dificultad para el análisis estático y pruebas unitarias

Los analizadores de código estático (como SonarQube o SpotBugs) suelen marcar como advertencia crítica (`warning`) cualquier llamada a un método no privado ni final desde un constructor. Esto se debe a que entorpece las pruebas unitarias: resulta difícil aislar o rastrear en qué momento y bajo qué reglas se transformaron las propiedades de un objeto durante la fase de instanciación.

Ejercicio 2: Programa empleado

Para el segundo ejercicio tuvimos que ingresar el nombre, apellido y el salario mensual para poder calcular su salario anual y cuando aumentamos un 10 por ciento el salario anual para posteriormente mostrarlo en pantalla, así con dos empleados:

```
Datos del primer empleado
Nombre: Luis
Apellido: Hernández
Salario mensual: 15000

Datos del segundo empleado
Nombre: Mariana
Apellido: López
Salario mensual: 20000
Salario anual de Luis Hernández : $180000.0
Salario anual de Mariana López : $240000.0

Con aumento del 10%:
Nuevo salario anual de Luis Hernández : $198000.0
Nuevo salario anual de Mariana López : $264000.0
```

Ejercicio 3: Programa Fecha

En el tercer ejercicio ingresamos la fecha que queremos mostrar siguiendo el formato mes/día/año, después podemos modificar la fecha inicial y ahora se muestra la fecha actualizada :

Ingrese la fecha inicial

Mes: 10

Día: 1

Año: 2026

Fecha registrada: 10/1/2026

Ingrese los nuevos valores de la fecha

Nuevo mes: 12

Nuevo día: 25

Nuevo año: 2026

Fecha actualizada: 12/25/2026

5. Conclusión

Se cumplieron los objetivos de la práctica, pues en primer lugar se consultó a un LLM sobre si era conveniente usar métodos modificadores dentro de los constructores para verificar la información, con base en la respuesta obtenida, se alcanzó el segundo objetivo al elaborar los programas requeridos, instanciando objetos como los empleados y aplicando el uso correcto de los modificadores para alterar atributos de forma independiente a la inicialización.

Referencias

- [1] DataCamp. *Constructores Java*. <https://www.datacamp.com/es/doc/java/constructors>. Publicado: 2021.
- [2] DataCamp. *Java Modifiers*. <https://www.datacamp.com/es/doc/java/modifiers>. Publicado: 2026.
- [3] W3schools. *Java Class Attributes*. https://www.w3schools.com/java/java_class_attributes.asp. Publicado: 2020.